CNN 합성곱 신경망

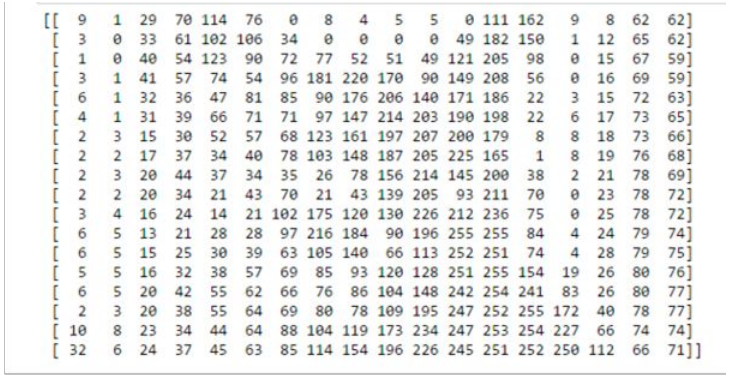
Convolution Neural Network

컴퓨터는 그림을 어떻게 읽는 걸까?

세로
100

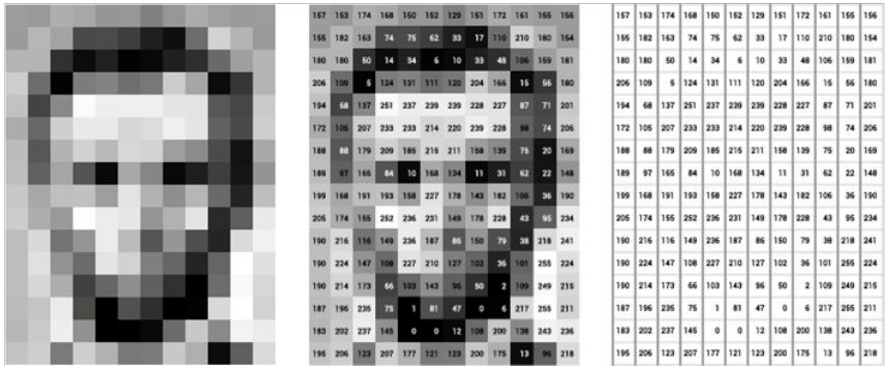


가로
250



```
[[  9   1  29  70 114  76   0   8   4   5   5   0 111 162   9   8  62  62]
 [  3   0  33  61 102 106  34   0   0   0   0  49 182 150   1  12  65  62]
 [  1   0  40  54 123  90  72  77  52  51  49 121 205  98   0  15  67  59]
 [  3   1  41  57  74  54  96 181 220 170  90 149 208  56   0  16  69  59]
 [  6   1  32  36  47  81  85  90 176 206 140 171 186  22   3  15  72  63]
 [  4   1  31  39  66  71  71  97 147 214 203 190 198  22   6  17  73  65]
 [  2   3  15  30  52  57  68 123 161 197 207 200 179   8   8  18  73  66]
 [  2   2  17  37  34  40  78 103 148 187 205 225 165   1   8  19  76  68]
 [  2   3  20  44  37  34  35  26  78 156 214 145 200  38   2  21  78  69]
 [  2   2  20  34  21  43  70  21  43 139 205  93 211  70   0  23  78  72]
 [  3   4  16  24  14  21 102 175 120 130 226 212 236  75   0  25  78  72]
 [  6   5  13  21  28  28  97 216 184  90 196 255 255  84   4  24  79  74]
 [  6   5  15  25  30  39  63 105 140  66 113 252 251  74   4  28  79  75]
 [  5   5  16  32  38  57  69  85  93 120 128 251 255 154  19  26  80  76]
 [  6   5  20  42  55  62  66  76  86 104 148 242 254 241  83  26  80  77]
 [  2   3  20  38  55  64  69  80  78 109 195 247 252 255 172  40  78  77]
 [ 10   8  23  34  44  64  88 104 119 173 234 247 253 254 227  66  74  74]
 [ 32   6  24  37  45  63  85 114 154 196 226 245 251 252 250 112  66  71]]
```

가로
250

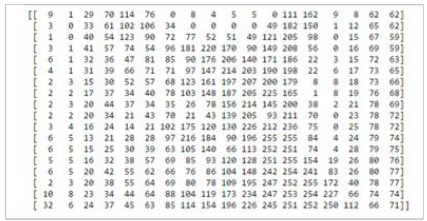


- 흑백 사진: 2차원 행렬 (가로 x 세로)
- RGB 사진: 3차원 텐서 (가로 x 세로 x 3)

세로
100



가로
250

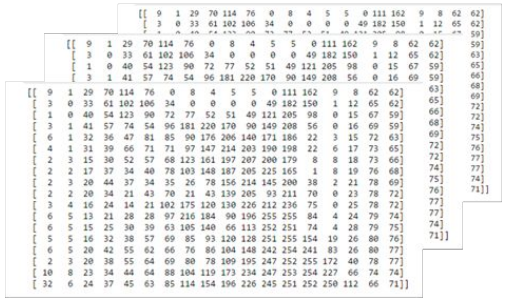


250 x 100

세로
80



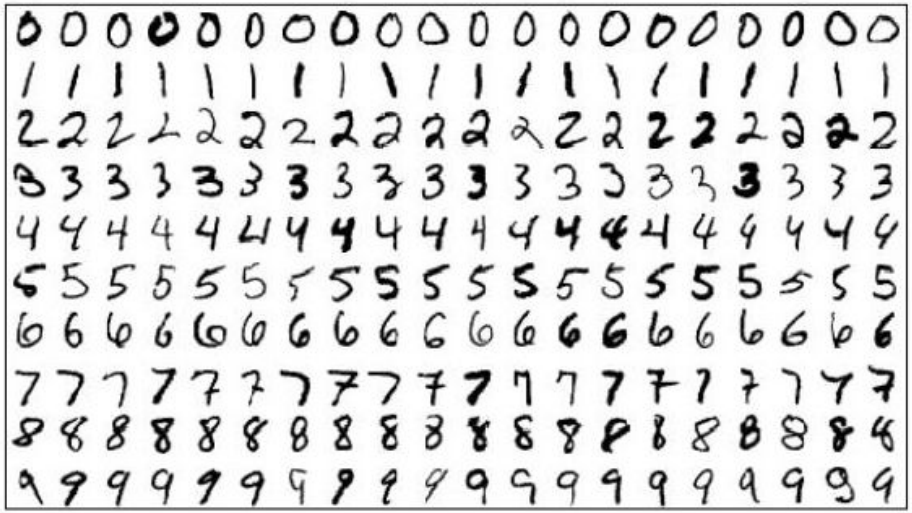
가로
250



250 x 80 x 3

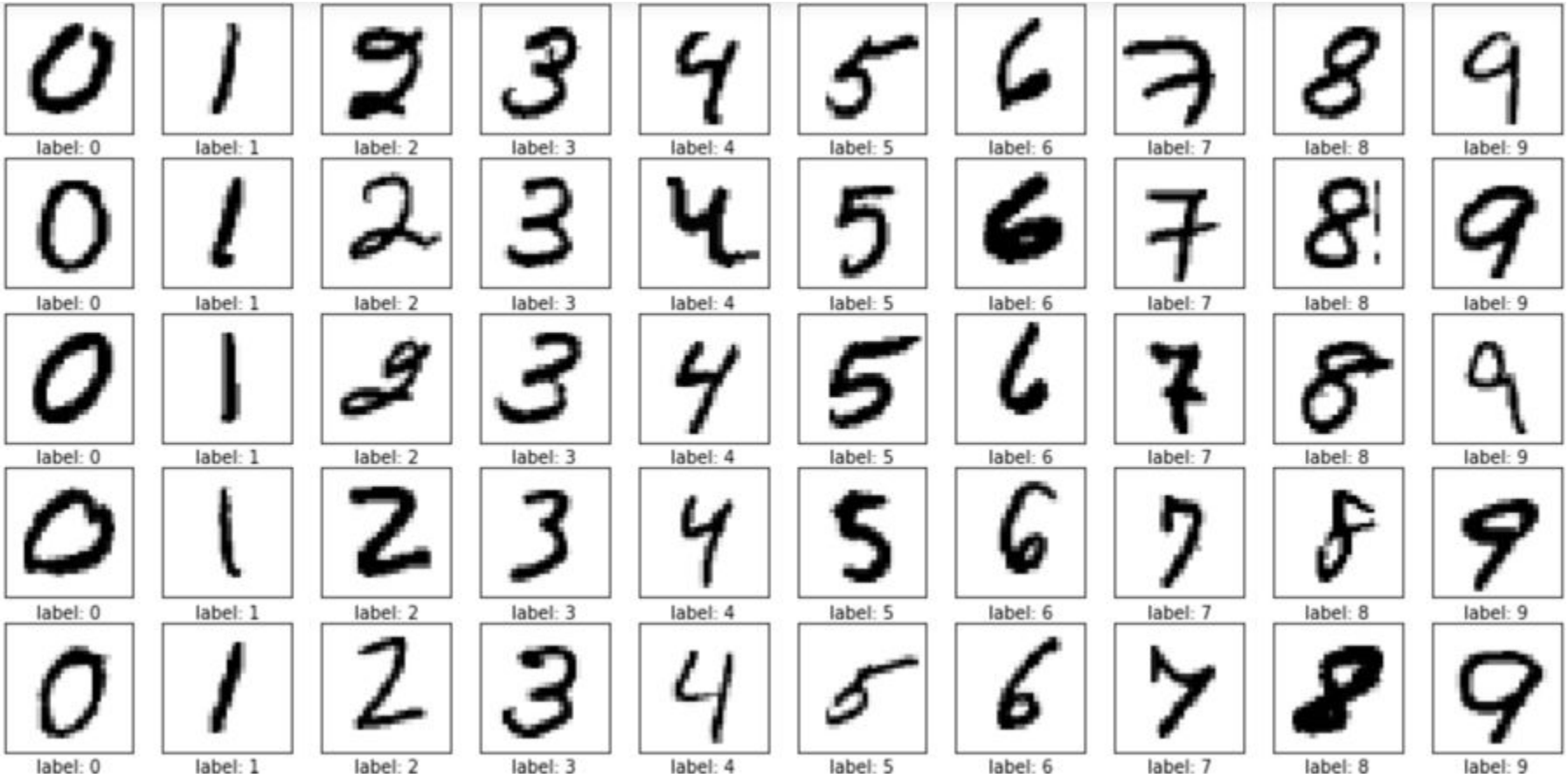


MNIST Dataset



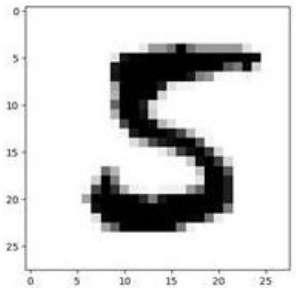
손으로 쓴 숫자(0부터 9까지)를 분류하기 위해 널리 쓰이는 표준 머신러닝·컴퓨터 비전 벤치마크 데이터셋

전체 이미지 수: 총 70,000개 (28×28 픽셀, 흑백/그레이스케일 이미지)



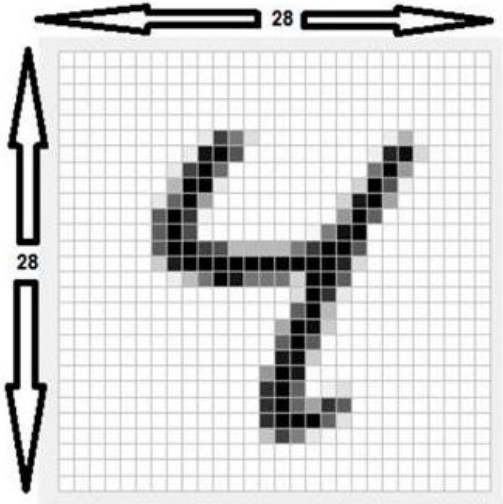
https://docs.ultralytics.com/ko/datasets/classify/mnist

6314번 이미지



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA	AB
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0	0.1647	0.4902	0.4902	0.6157	0.9961	0.6275	0.4902	0.4902	0.4902	0.4902	0.4902	0.2118	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0.1922	0.9216	0.9725	0.9804	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9804	0.949	0	0	0
7	0	0	0	0	0	0	0	0	0	0.898	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.7216	0.5922	0.9686	0.2824	0	0	0
8	0	0	0	0	0	0	0	0	0	0.9216	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9098	0.5843	0.502	0.0745	0.0275	0.0078	0.0706	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0.4627	0.9922	0.9922	0.9922	0.9922	0.8824	0.2157	0.098	0.0784	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0.4118	0.9922	0.9922	0.9922	0.9922	0.2667	0	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0.6745	0.9922	0.9922	0.8824	0.2392	0.0314	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	0	0.4118	0.9922	0.9922	0.9333	0.3059	0	0	0	0	0	0	0	0	0	0	0	0	0	0
13	0	0	0	0	0	0	0	0	0	0.0863	0.7686	0.9922	0.9922	0.9294	0.3882	0.2118	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	0	0.0902	0.898	0.9922	0.9922	0.9922	0.9333	0.7686	0.4	0.0784	0	0	0	0	0	0	0	0	0
15	0	0	0	0	0	0	0	0	0	0	0	0.2235	0.4235	0.7608	0.8824	0.9412	0.9922	0.9922	0.7255	0.0784	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.1451	0.349	0.7882	0.9373	0.9922	0.8078	0.2588	0	0	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0.3255	0.9373	0.9922	0.7961	0.2667	0	0	0	0	0
18	0	0	0	0	0	0	0	0	0.6353	0.8922	0	0	0	0	0	0	0	0	0.349	0.9922	0.9922	0.7216	0	0	0	0	0	0
19	0	0	0	0	0	0	0	0.298	0.9765	0.5843	0	0	0	0	0	0	0	0	0.1804	0.9922	0.9922	0.8902	0	0	0	0	0	0
20	0	0	0	0	0	0	0.051	0.7804	0.9922	0.8353	0.4196	0.1059	0.0706	0	0.0824	0.1059	0.1059	0.4745	0.6824	0.9922	0.9922	0.8902	0	0	0	0	0	0
21	0	0	0	0	0	0	0.451	0.9922	0.9922	0.9922	0.9922	0.9922	0.8588	0.5882	0.9098	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.8902	0	0	0	0	0	0
22	0	0	0	0	0	0	0	0.9451	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9608	0.5804	0	0	0	0	0	0
23	0	0	0	0	0	0	0	0.2118	0.9686	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.9725	0.9686	0.6235	0.1647	0	0	0	0	0	0	0
24	0	0	0	0	0	0	0	0	0.4627	0.6902	0.9922	0.9922	0.9922	0.9922	0.9922	0.9922	0.6078	0.1373	0	0	0	0	0	0	0	0	0	0
25	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
26	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
28	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- 28x28 = 784개의 픽셀
- 10가지 숫자



$$
\begin{bmatrix}
x_0 w_{A0} + x_1 w_{A1} + x_2 w_{A2} + \cdots + x_{783} w_{A783} = y_0 \\
x_0 w_{B0} + x_1 w_{B1} + x_2 w_{B2} + \cdots + x_{783} w_{B783} = y_1 \\
x_0 w_{C0} + x_1 w_{C1} + x_2 w_{C2} + \cdots + x_{783} w_{C783} = y_2 \\
x_0 w_{D0} + x_1 w_{D1} + x_2 w_{D2} + \cdots + x_{783} w_{D783} = y_3 \\
\vdots \\
x_0 w_{J0} + x_1 w_{J1} + x_2 w_{J2} + \cdots + x_{783} w_{J783} = y_9
\end{bmatrix}
$$

총 몇 개의 weight이 학습을?

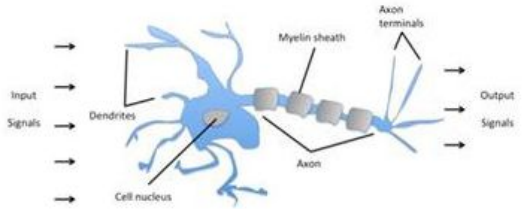
$$
\begin{bmatrix}
x_0 w_{A0} + x_1 w_{A1} + x_2 w_{A2} + \cdots + x_{783} w_{A783} = y_0 \\
x_0 w_{B0} + x_1 w_{B1} + x_2 w_{B2} + \cdots + x_{783} w_{B783} = y_1 \\
x_0 w_{C0} + x_1 w_{C1} + x_2 w_{C2} + \cdots + x_{783} w_{C783} = y_2 \\
x_0 w_{D0} + x_1 w_{D1} + x_2 w_{D2} + \cdots + x_{783} w_{D783} = y_3 \\
\vdots \\
x_0 w_{J0} + x_1 w_{J1} + x_2 w_{J2} + \cdots + x_{783} w_{J783} = y_9
\end{bmatrix}
$$

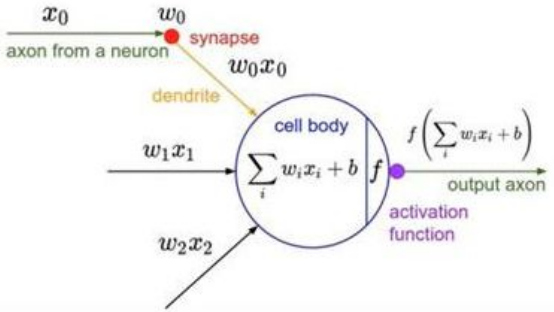
위의 계산식이 눈에 익숙한가요?

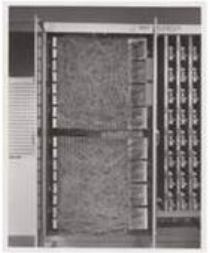
독립변수 $X_{0\sim783}$가 784개, 종속변수 $y_{0\sim9}$가 10개인 **다중 분류 문제**



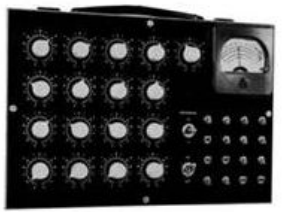
Multilayer Perceptrons







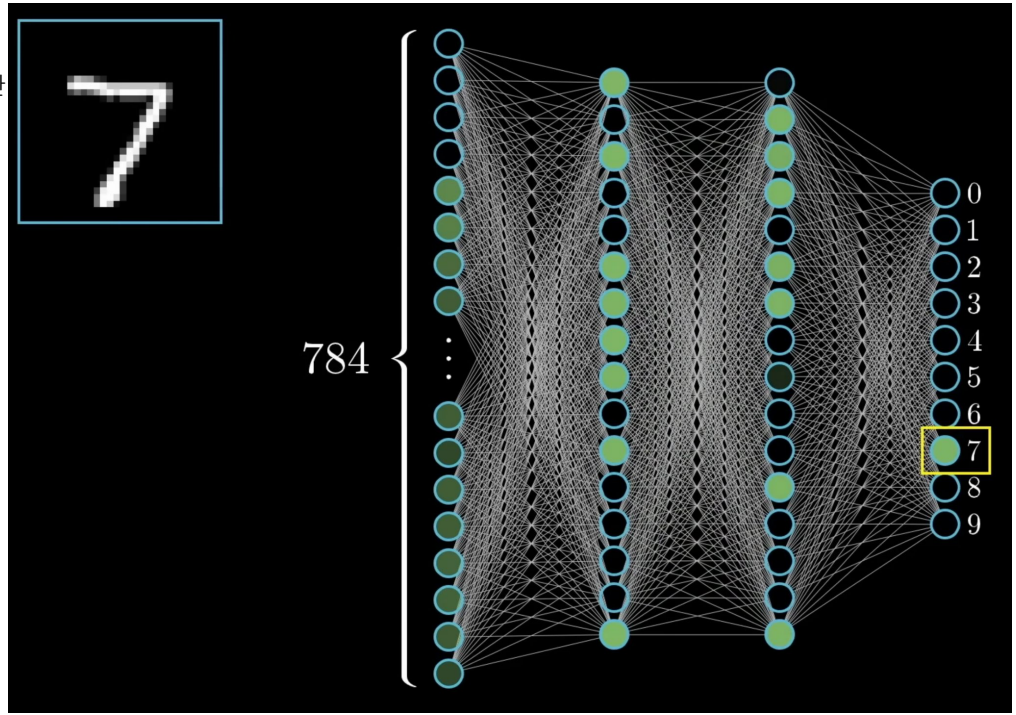
Frank Rosenblatt, ~1957: Perceptron



Widrow and Hoff, ~1960: Adaline/Madaline

MLP(다층 퍼셉트론) DNN 모델로 이미지 분류하기

다층 퍼셉트론은 입력 데이터를 1차원(Flatten)으로 쭉 펴서 사용
은닉층을 거치면서 이미지의 모든 픽셀위치에 대한 가중치를 계산
DNN 다중분류 모델



DNN 이미지 분류의 단점

모든 층의 노드가 빽빽하게 연결(Fully Connected)되어 있음
연결이 많아 파라미터가 매우 많음. 계산량이 많음
이미지를 1차원으로 펴는 과정에서 위치/공간 정보가 망가짐.
이미지의 위치가 바뀌면 처음부터 다시 배워야 할 정도로 취약함



딥러닝으로 이미지를 분류하는 것에 대한 개념 이해 유튜브 영상 [재생시간 18:40]
https://www.youtube.com/watch?v=aircAruvnKk

이미지의 윤곽만 알아도 이미지를 분류할 수 있다.

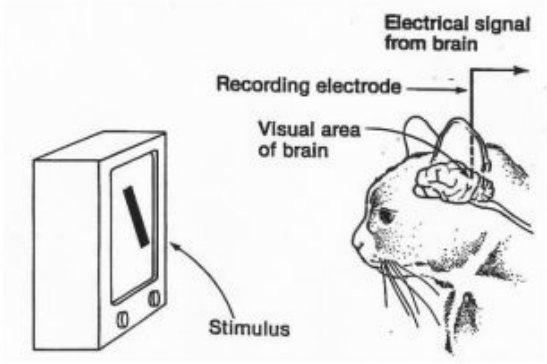
이미지의 feature가 될 수 있는 요소는 굉장히 많지만 그 중에서 대표적인 것이 **형태**일 것이다. 물체의 윤곽부분만 보더라도 우리는 물체가 무엇인지 구분하는데 큰 정보를 얻을 수 있음. 아래 사진에서도 오른쪽의 윤곽선만 보더라도 우리는 물체를 쉽게 가늠해볼 수 있음.

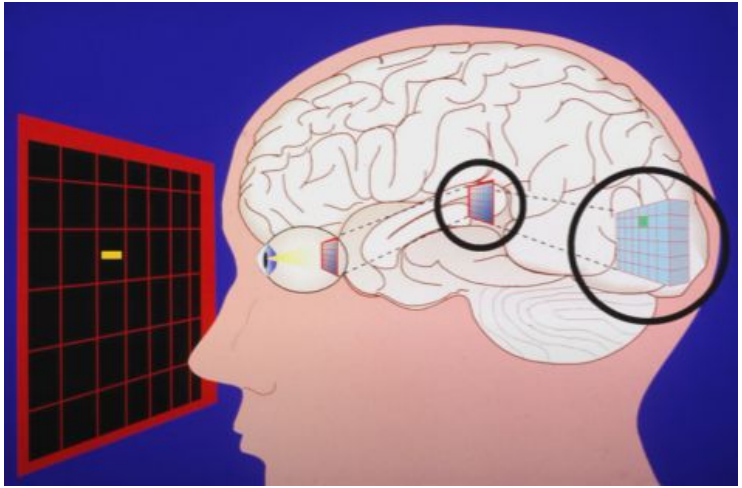


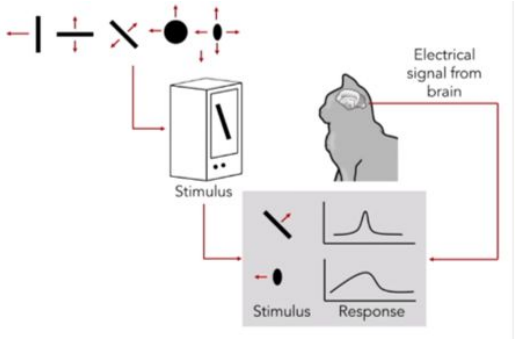


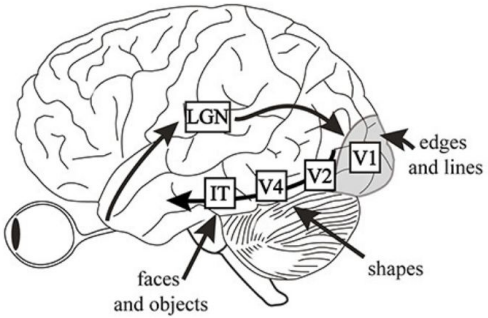
CNN 모델의 등장 배경. 고양이 시각피질 검사

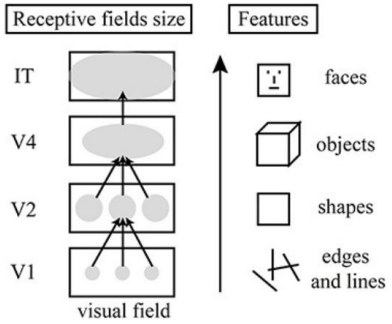
<고양이의 시각피질 검사>



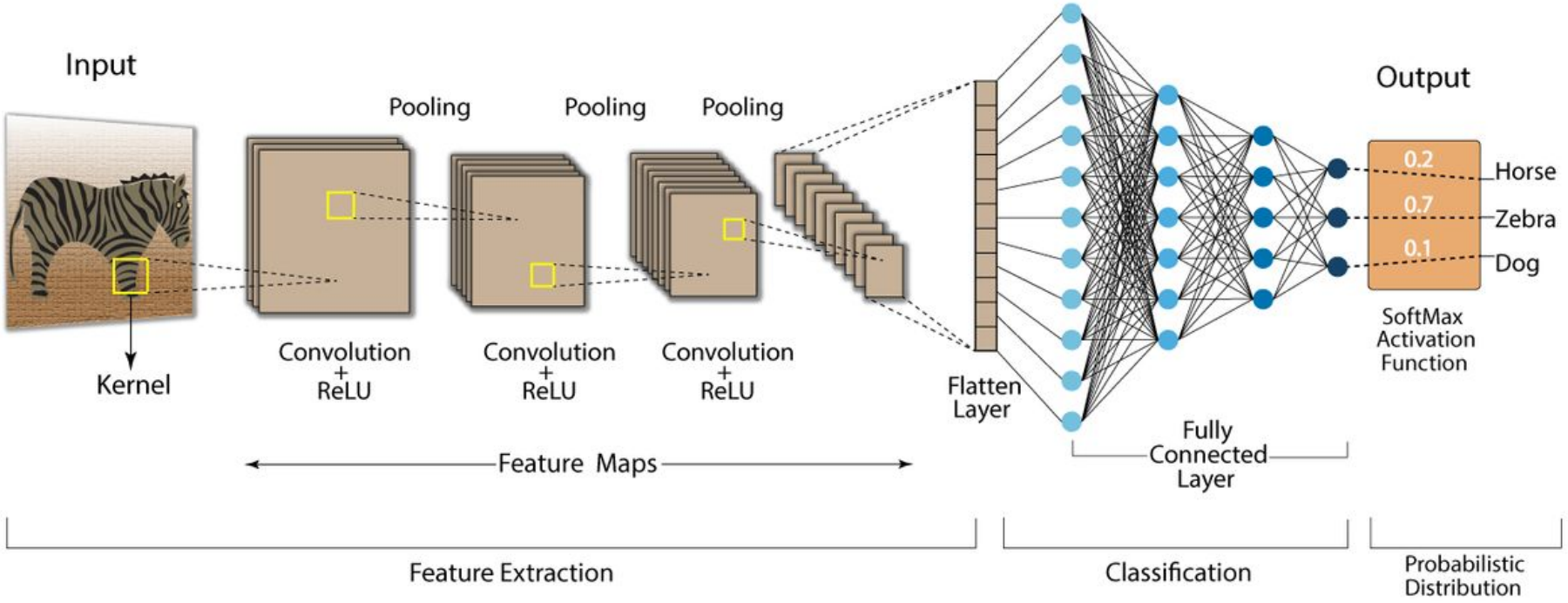








Convolution Neural Network (CNN)

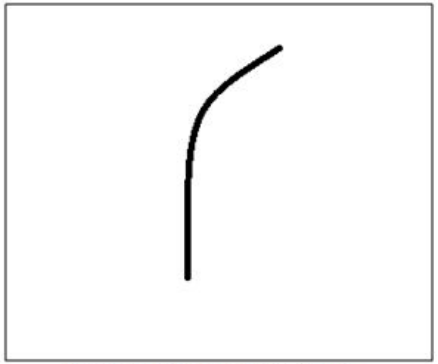


윤곽을 알아내는 Convolution Layer

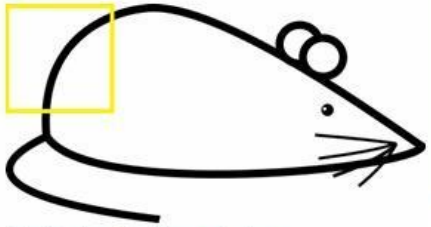
CNN에서 물체의 윤곽을 알아내기 위해서 제일 처음 하는 작업은 edge filter를 사용한 합성곱 연산임. Convolution feature filter는 모든 layer에서 같은 크기로 고정되어 있음. (보통은 3x3 size로 구성) 그렇기 때문에 처음에 들어오는 이미지 크기는 convolution feature filter 크기에 비해 상대적으로 크기 때문에 filter가 이미지를 커버할 수 있는 부분이 선과 같은 단순한 정보임. (이러한 단순한 모양들을 edge라고 부름).

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter



Visualization of the filter on the image



Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

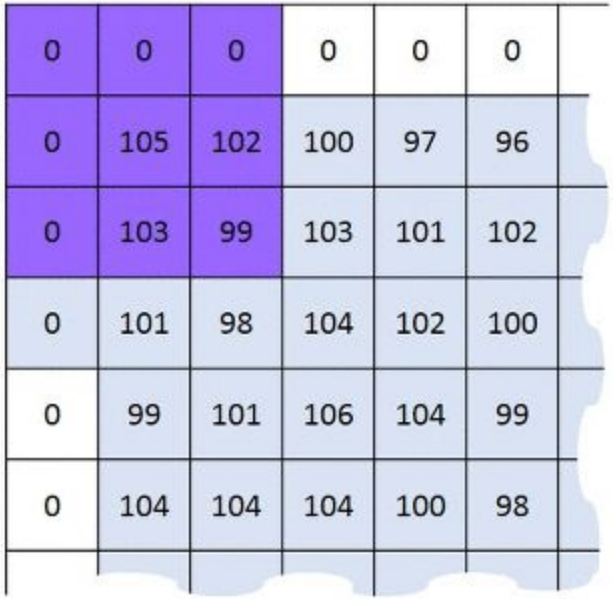
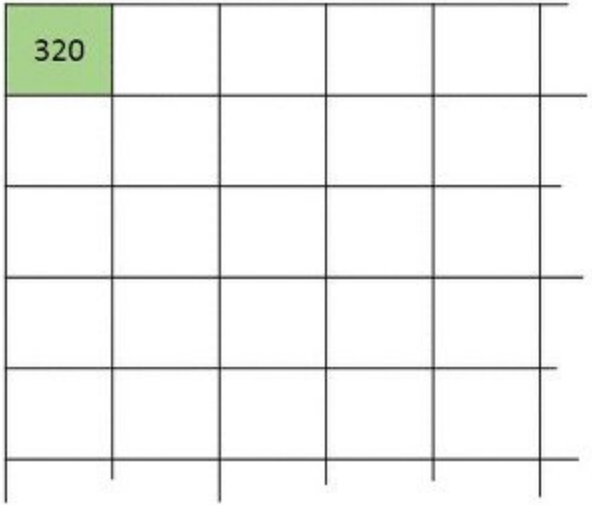


Image Matrix

Kernel Matrix

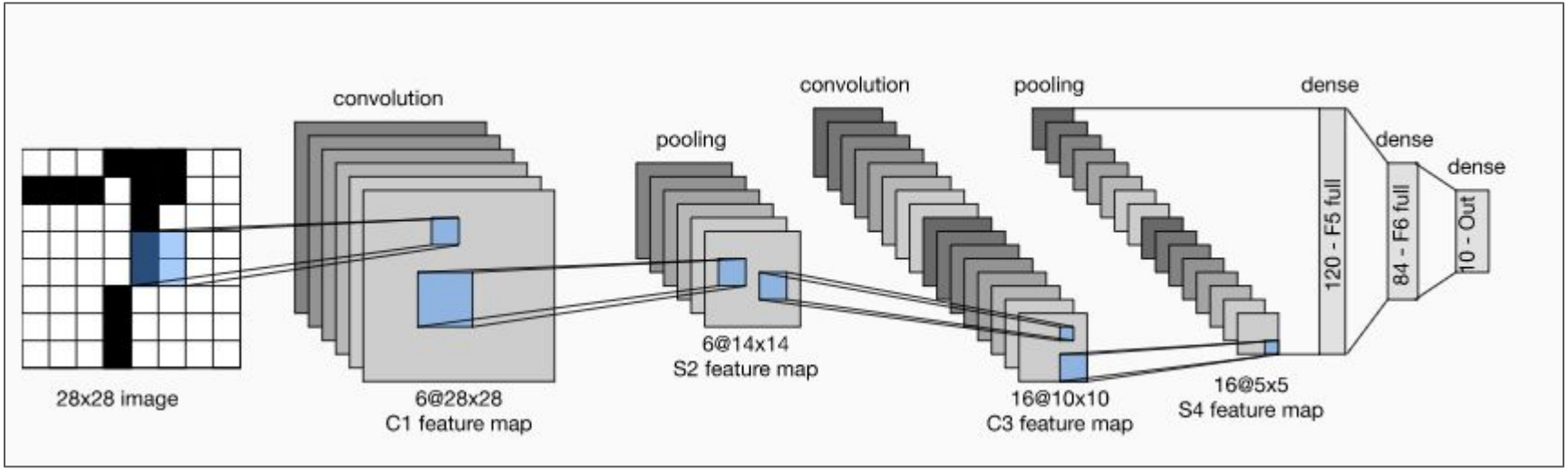
0	-1	0
-1	5	-1
0	-1	0

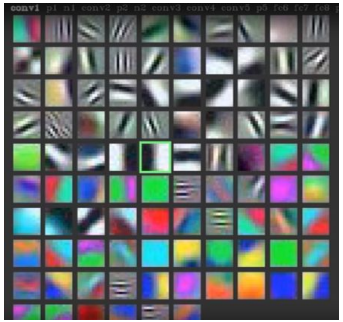


Output Matrix

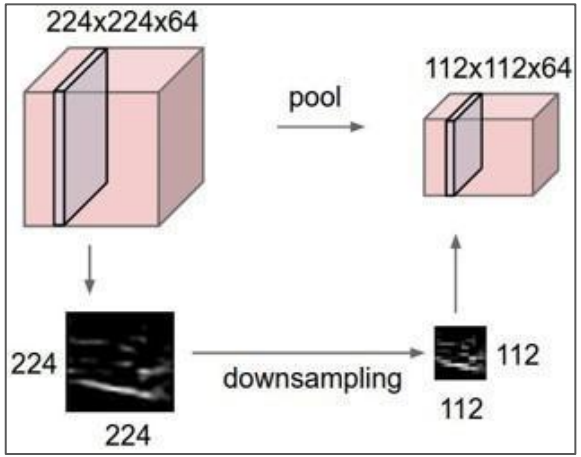
$$0*0+0*-1+0*0$$
$$+0*-1+105*5+102*-1$$
$$+0*0+103*-1+99*0 \quad = 320$$

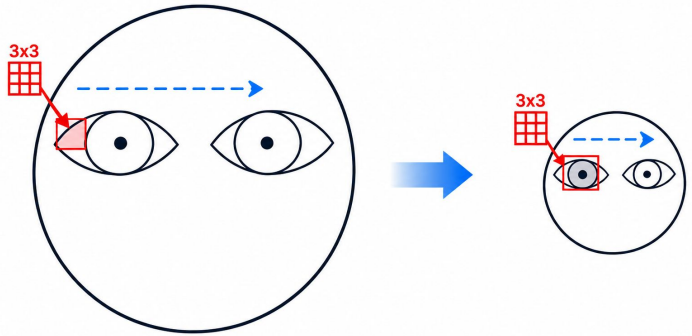
Convolution with horizontal and vertical strides = 1


convolution
pooling
convolution
pooling
dense
dense
dense
120 - F5 full
84 - F6 full
10 - Out
28x28 image
6@28x28
C1 feature map
6@14x14
S2 feature map
16@10x10
C3 feature map
16@5x5
S4 feature map



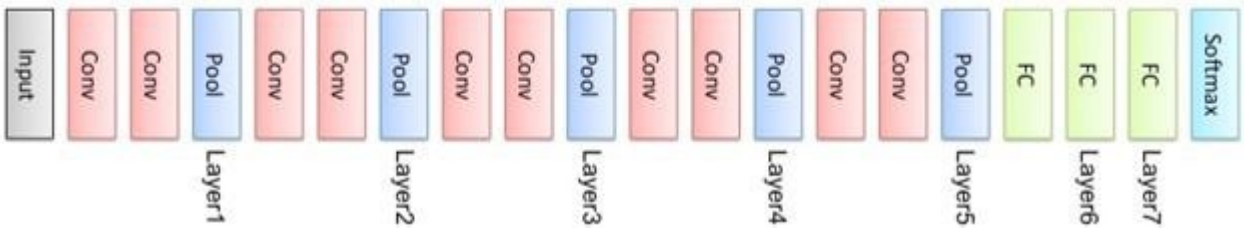
96개의 edge
필터

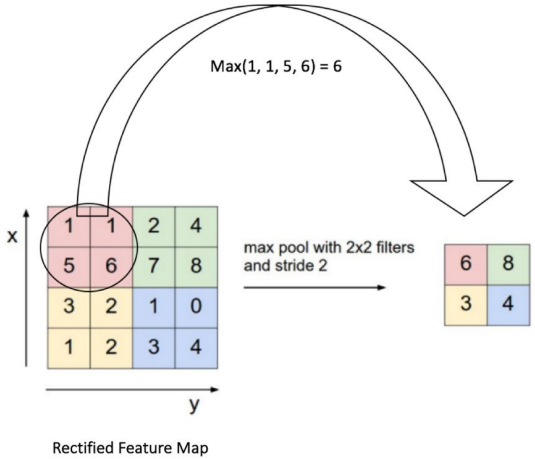

224x224x64
pool
112x112x64
224
224
downsampling
112
112

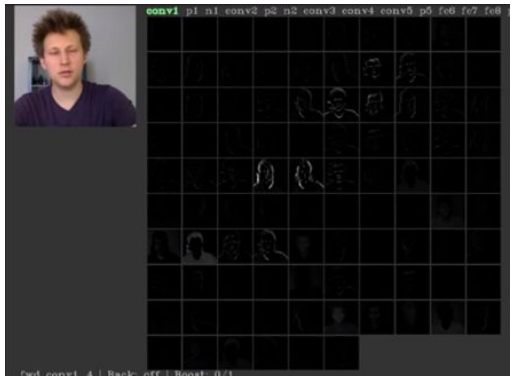

3x3
3x3

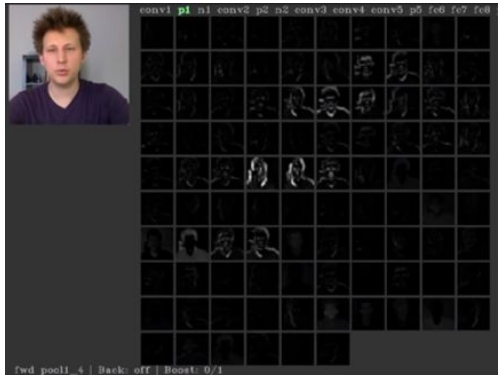
풀링층 Pooling Layer

CNN 구조를 보면 convolution layer (Conv) 사이에 Pooling layer (pool)가 사이사이에 배치하고 있음.









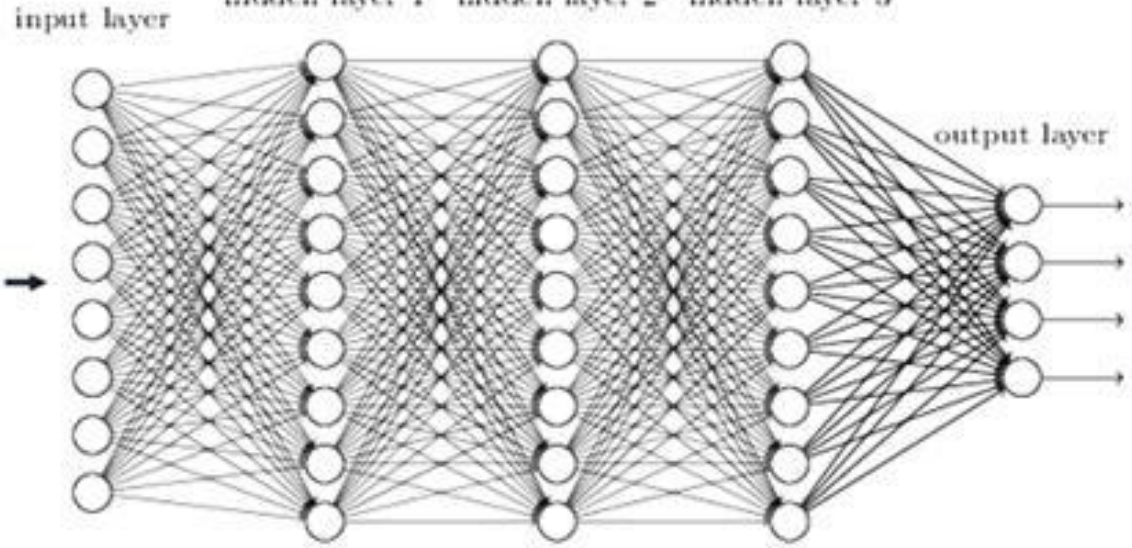
Max pooling은 말그대로 특정영역에서 가장 큰 값을 꺼내어 쓰는것임.
두 사진을 보면 모양은 그대로인데 pooling결과 좀 더 feature들이 뚜렷해지는 것을 볼 수 있음

Deep neural networks learn hierarchical feature representations



hidden layer 1 hidden layer 2 hidden layer 3

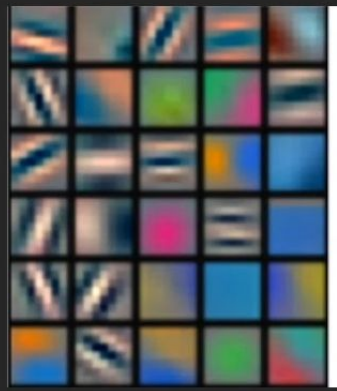




차 사진을 신경망에 집어넣었더니..

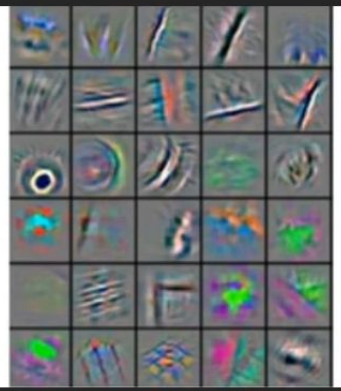


layer1

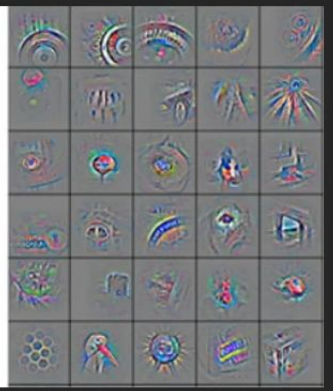


직선, 곡선

layer2



layer3

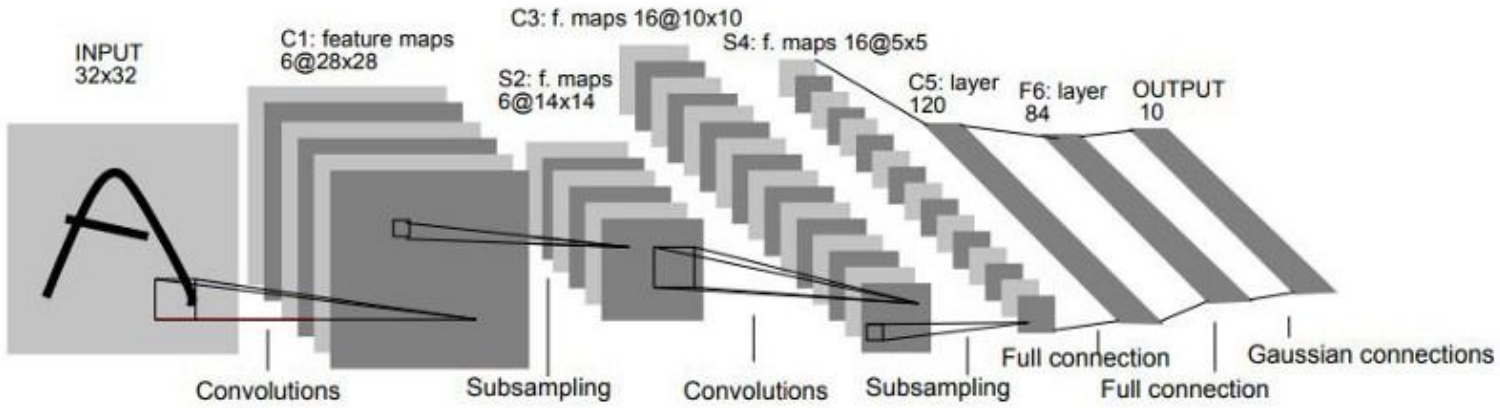


바퀴, 백미러, 손잡이

대표적인 CNN 이미지분류 모델들

LeNet-5 (1998) — 현대 CNN의 출발점

Yann LeCun 연구팀이 개발한 초기 CNN의 대표 모델로, 손글씨 숫자 인식에서 CNN의 실용성을 입증



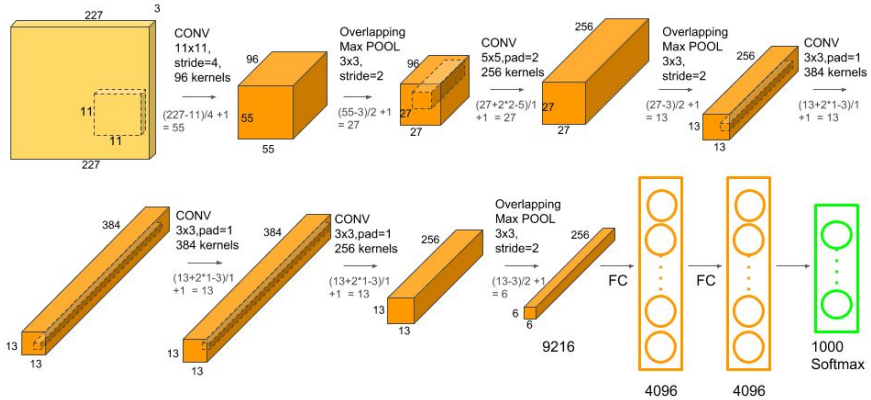
핵심 특징

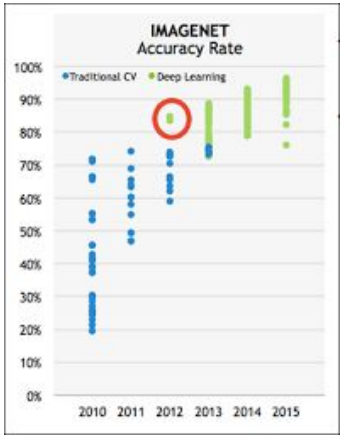
- 역사적 의의: 기존 합성곱·풀링 개념을 통합하여 실용적으로 동작하는 CNN 구조를 제시
- 기술적 특징: 합성곱(특징 추출), 풀링(차원 축소), 완전연결층(분류)을 계층적으로 구성
- 설계 철학: 필터 크기·개수·층 구성은 수학적으로 증명된 최적해가 아닌 반복적인 실험과 경험을 통해 결정

LeNet은 CNN의 핵심 요소들을 하나의 성공적인 아키텍처로 구현했으며, 이후 CNN 모델 발전의 토대가 되었다.

AlexNet (2012) — 딥러닝과 CNN의 대중화를 이끈 혁신

Alex Krizhevsky 등이 개발한 CNN 모델로, 2012년 ImageNet 대회에서 압도적인 성능으로 우승하며 딥러닝의 가능성을 입증





핵심 특징

- GPU 병렬 연산: 2개의 GPU를 활용하여 대규모 데이터 학습 및 깊은 신경망 훈련 실현
- ReLU 활성화 함수: 기존 tanh·sigmoid 대비 학습 속도를 크게 향상
- Dropout 적용: 일부 뉴런을 무작위로 비활성화하여 과적합 완화 및 일반화 성능 개선
- 심층 네트워크: 5개 합성곱층(Conv) + 3개 완전연결층(FC)로 구성된 8개 학습층

AlexNet Top-5 오류율

15.3%

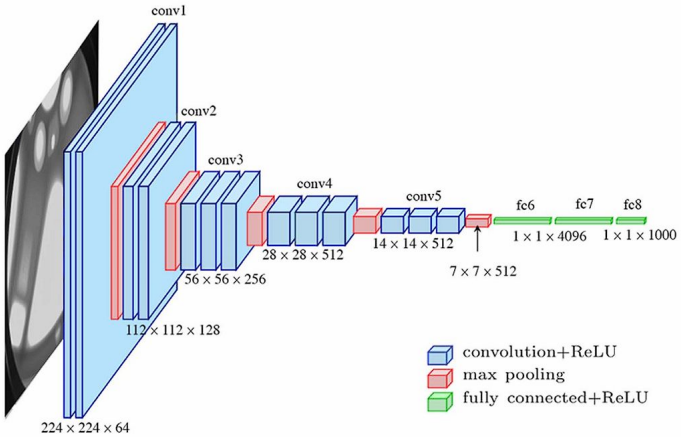
2위 모델 Top-5 오류율

26.2%

기존 이미지 인식 기법 대비 획기적인 성능 향상을 달성하며, CNN이 컴퓨터 비전의 주류 기술로 자리 잡는 전환점이 됨.

VGG (2014) — 단순함과 깊이의 중요성을 입증한 CNN

옥스퍼드대학교VGG 연구팀이 개발한 CNN 모델로, 작은 3×3 필터를 반복적으로 쌓는 단순하고 일관된 구조를 통해 네트워크 깊이의 중요성을 입증



VGG-16 학습층

16층

13 Conv + 3 FC

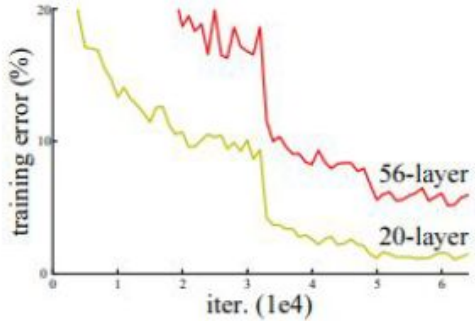
핵심 특징

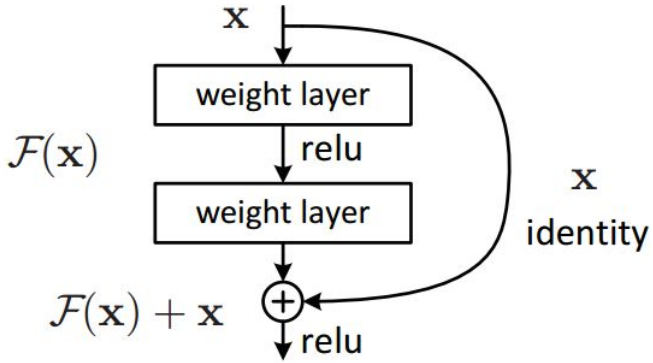
- 3×3 소형 필터: AlexNet의 다양한 크기 필터 대신 작은 3×3 필터를 일관되게 사용
- 깊은 신경망: VGG-16(16개 학습층), VGG-19(19개 학습층) 등 네트워크를 깊게 구성하여 특징 추출 능력 강화
- 단순하고 규칙적인 구조: Conv(3×3) 반복 → Max Pooling(2×2) → FC 구조 채택
- 효율적인 특징 추출: 작은 필터를 여러 번 중첩하여 수용 영역을 넓히고 비선형성을 증가시켜 표현력 향상

AlexNet보다 단순하고 깊은 구조로 높은 이미지 인식 성능을 달성했으며, 이후 CNN 아키텍처 설계의 중요한 기반이 됨.

ResNet (2015) — 잔차 연결로 초심층 CNN의 한계를 극복

Microsoft Research의 Kaiming He 연구팀이 개발한 CNN 모델로, 잔차 연결(Residual Connection)을 도입하여 매우 깊은 신경망의 효과적인 학습을 실현





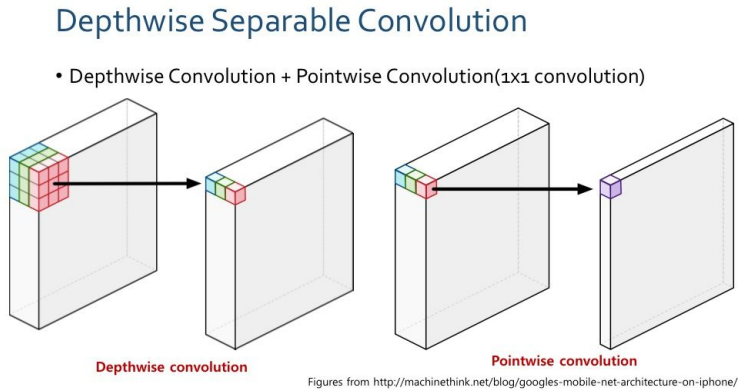
핵심 특징

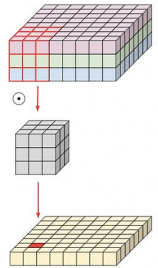
- 기존 CNN의 한계: 네트워크가 깊어질수록 기울기 소실 및 최적화의 어려움으로 학습 성능이 저하되는 문제 발생
- 잔차 연결(Residual Connection): 입력을 여러 층을 건너뛰어 출력에 직접 더하는 Skip Connection 구조 도입
- 효율적인 학습: 전체 변환 대신 잔차(Residual)를 학습하여 기울기 전달 및 최적화 문제 완화
- 초심층 네트워크: ResNet-50, ResNet-101, ResNet-152 등 100층 이상의 깊은 신경망 학습 실현

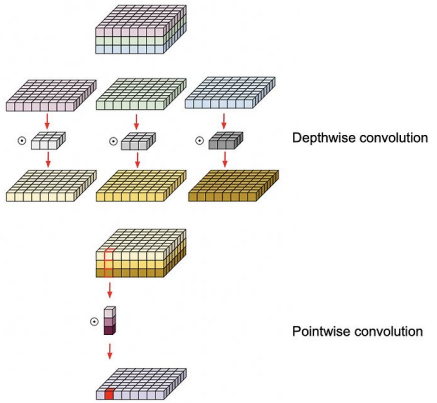
초심층 신경망의 학습 문제를 획기적으로 개선하여 이미지 인식 성능을 높였으며, 이후 딥러닝 아키텍처의 핵심 설계 기법으로 자리 잡음.

MobileNet (2017) — 경량화와 연산 효율성을 혁신한 CNN

Google 연구팀이 개발한 경량 CNN 모델로, Depthwise Separable Convolution을 도입하여 정확도를 최대한 유지하면서 연산량과 모델 크기를 획기적으로 [illegible]







Standard convolution layer

Depthwise separable convolution

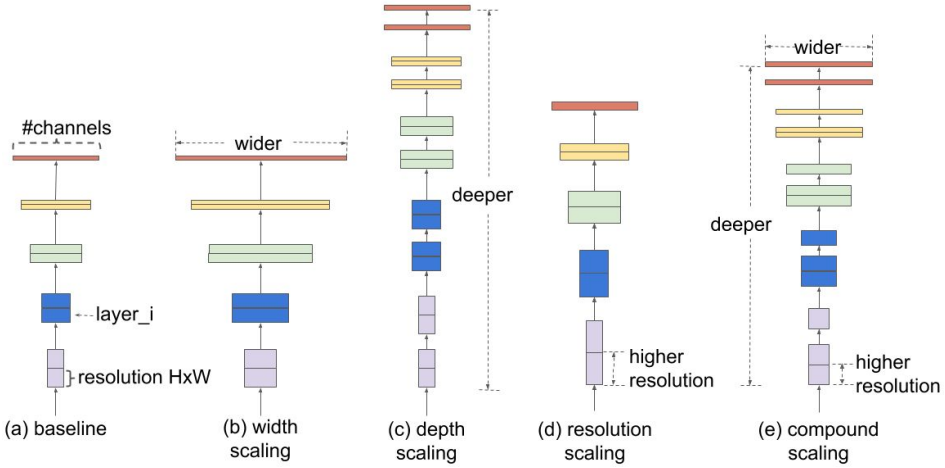
핵심 특징

- 경량 CNN 구조: 모바일 및 임베디드 기기의 제한된 연산 자원과 메모리 환경에 최적화
- Depthwise Convolution: 입력 채널별로 독립적인 합성곱을 수행하여 연산량 감소
- Pointwise Convolution: 1×1 합성곱을 통해 채널 간 정보를 결합하고 새로운 특징 생성
- 연산 효율성: 기존 표준 합성곱 대비 약 8~9배 적은 연산량으로 유사한 특징 추출 성능 구현
- 후속 발전: MobileNetV2의 Inverted Residual·Linear Bottleneck, V3의 추가 최적화 기법으로 경량화 발전

고성능 GPU 중심의 CNN을 모바일·엣지 디바이스에서도 효율적으로 추론할 수 있도록 발전시키며, 딥러닝 연구의 방향을 성능 중심에서 경량화·효율성 중심으로 확장. 모바일·엣지 AI 시대의 기반을 마련한 경량 CNN 모델

EfficientNet (2019) — 균형 잡힌 확장으로 CNN 효율성 극대화

Google 연구팀이 개발한 CNN 모델로, 깊이(Depth)·너비(Width)·해상도(Resolution)를 균형 있게 확장하는 Compound Scaling 기법을 통해 정확도와 연산 효율성을 동시에 향상



핵심 특징

- Compound Scaling: 네트워크의 깊이·너비·입력 해상도를 일정한 비율로 동시에 확장하는 기법 도입
- Depth·Width·Resolution 최적화: 층 수, 채널 수, 이미지 해상도의 균형을 통해 효율적인 특징 추출 실현
- 효율적인 네트워크 구조: MBConv, Squeeze-and-Excitation, Swish(SiLU) 활성화 함수 등을 활용하여 성능 향상
- 다양한 모델 규모: EfficientNet-B0부터 B7까지 연산 자원과 정확도 요구에 맞는 모델 제공
- 후속 발전: EfficientNetV2에서 학습 속도, 파라미터 효율성 및 메모리 사용량 개선

EfficientNet은 Compound Scaling(깊이, 너비, 해상도) 을 통해 모델의 크기와 연산량 대비 높은 정확도를 달성., CNN 아키텍처의 효율적인 확장 방법을 정립

이젠 비전 트랜스포머의 시대

Vision Transformer (ViT, 2020) — CNN을 넘어선 새로운 이미지 인식 모델

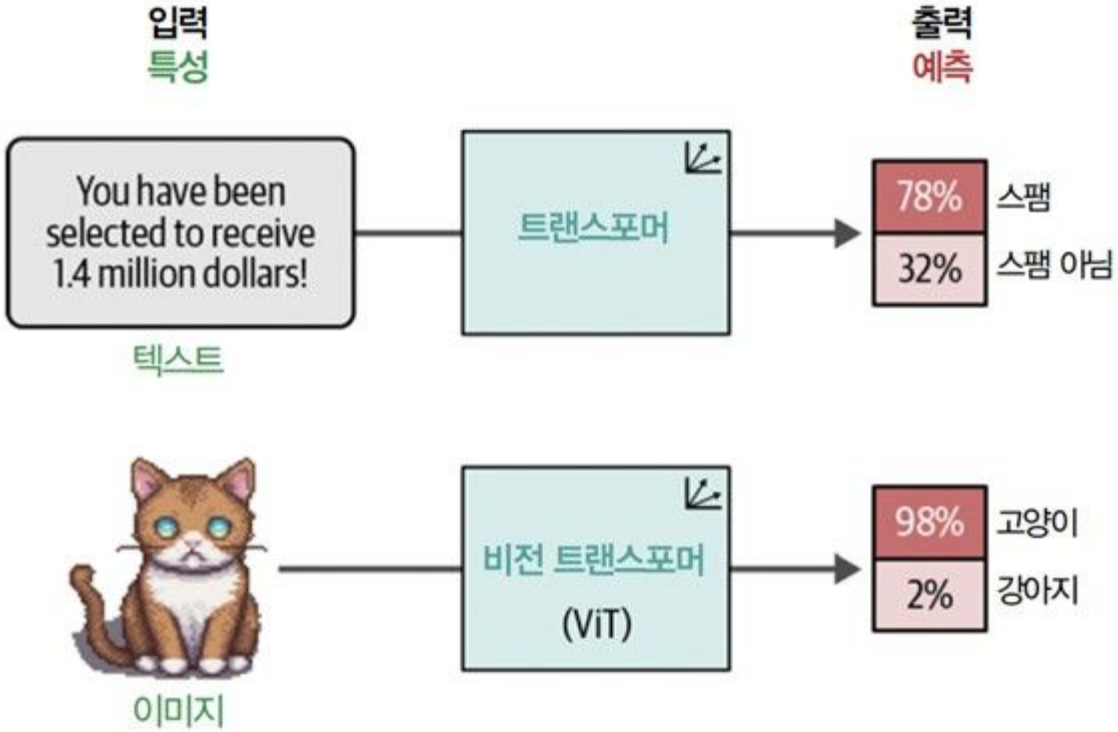
Google Research가 제안한 이미지 인식 모델로, 자연어 처리(NLP)에서 사용하던 Transformer 구조를 컴퓨터 비전에 적용하여 CNN 없이도 높은 이미지 분류 성능을 달성

핵심 특징

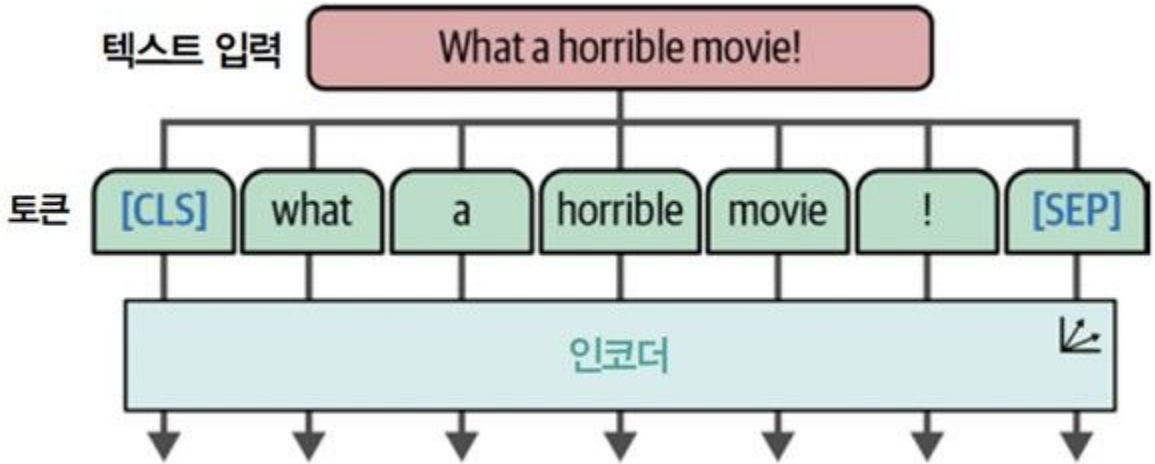
- 이미지 패치 분할(Patch Embedding): 이미지를 16×16 등 작은 패치로 나누고, 각 패치를 Transformer가 처리할 수 있는 벡터(Token)로 변환
- Self-Attention: 이미지의 각 패치가 다른 패치와의 관계를 학습하여 이미지 전체의 특징을 파악
- Position Embedding: 패치의 위치 정보를 추가하여 이미지의 공간적 구조를 학습
- Transformer Encoder: Multi-Head Self-Attention과 MLP 계층을 반복하여 이미지 특징 추출
- CNN 의존성 제거: 합성곱 필터 없이도 이미지 특징을 학습할 수 있음을 입증

대규모 사전학습을 통해 기존 고성능 CNN과 경쟁하거나 이를 능가하는 성능을 달성하며, 이미지 인식 분야에서도 Transformer가 효과적임을 입증했다. 다만 데이터 요구량이 크고, 고해상도 이미지에서는 Attention 연산 비용이 증가할 수 있다.

공간 정보 : 합성곱 구조에 내재, 위치 임베딩 활용

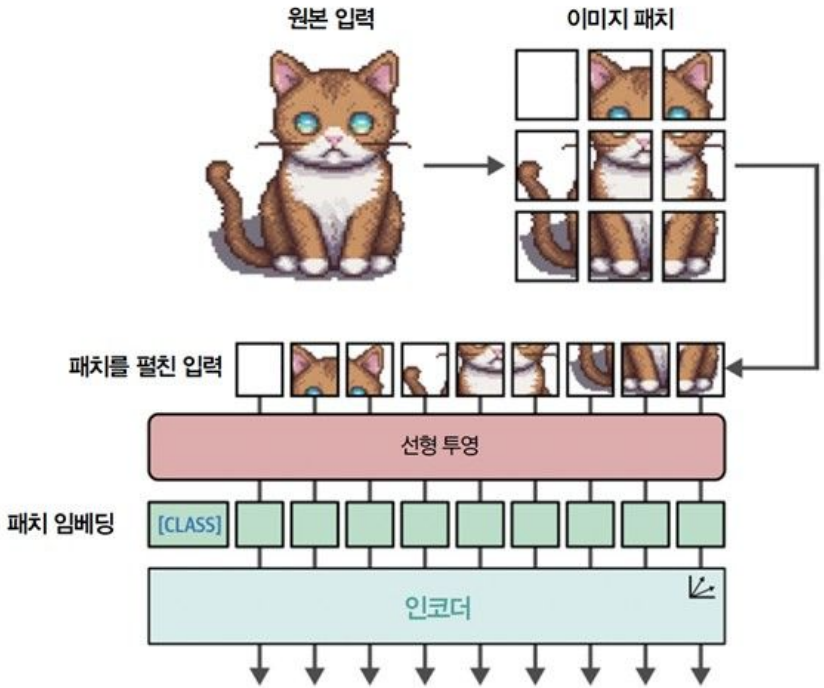

입력
특성
출력
예측
You have been selected to receive 1.4 million dollars!
텍스트
트랜스포머
78%
스팸
32%
스팸 아님
이미지
비전 트랜스포머
(ViT)
98%
고양이
2%
강아지

자연어 처리 트랜스포머



먼저 텍스트를 토크나이저를 사용해 토큰으로 바꾼 후 한 개 이상의 인코더에 통과시킴

비전 트랜스포머 ViT

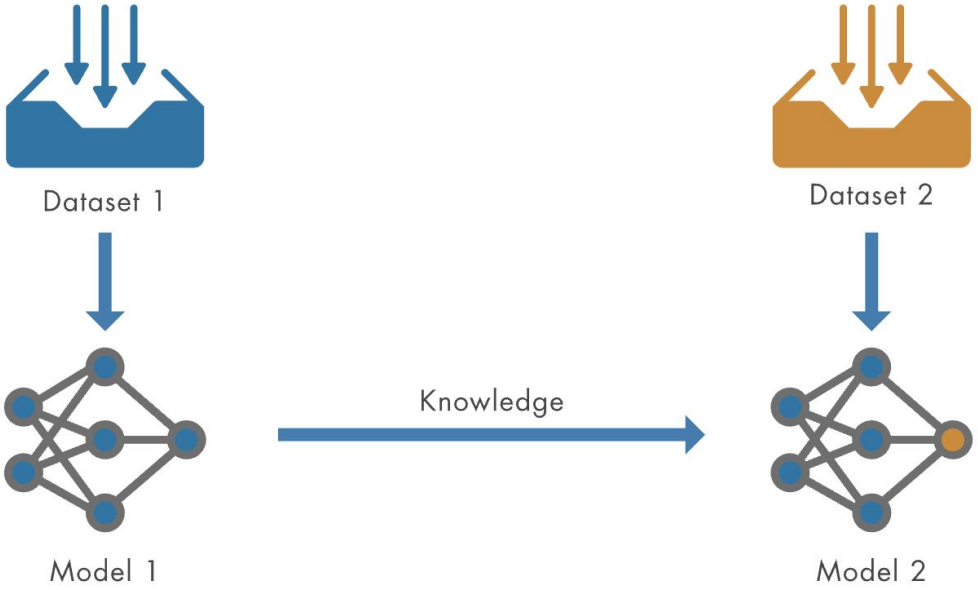


ViT의 핵심 알고리즘

이미지를 패치로 나누고 선형 투영한 후, 패치 임베딩이 인코더로 전달되어 텍스트 토큰처럼 처리

전이학습

Transfer Learning



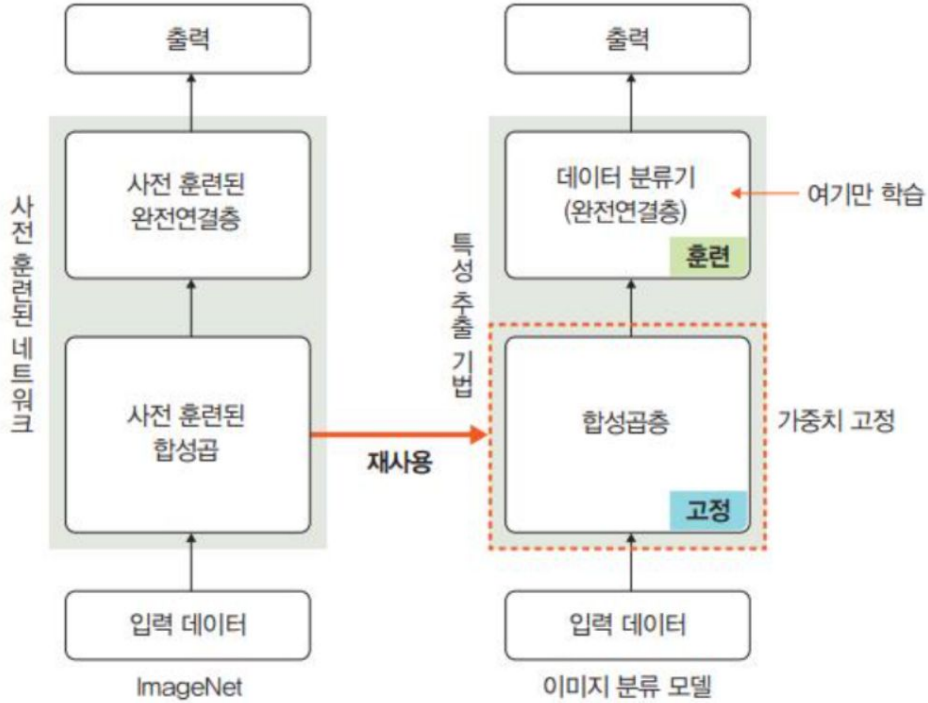
전이학습이란?

전이 학습은 어떤 작업에 대해 기존에 훈련된 모델을 사용해 유사한 작업을 수행하는 모델의 기초로 활용하는 딥러닝 접근법입니다.

전이 학습을 통해 신경망을 업데이트하고 재훈련시키는 것이 처음부터 신경망을 훈련시키는 것보다 빠르며 쉽습니다.

이 방식은 특히 학습 데이터가 부족한 상황에서 유용하며, 기존에 학습된 모델이 축적한 지식을 활용하여 새로운 문제에 더 빠르고 효과적으로 접근할 수 있도록 합니다.

전이 학습은 이미지 분류, 객체 검출, 음성 인식 및 기타 응용 사례에서 사용됩니다.



특성 추출기법은 데이터에서 의미 있는 정보를 추출하고, 원래 데이터를 더 적은 차원의 특성으로 변환하는 기법

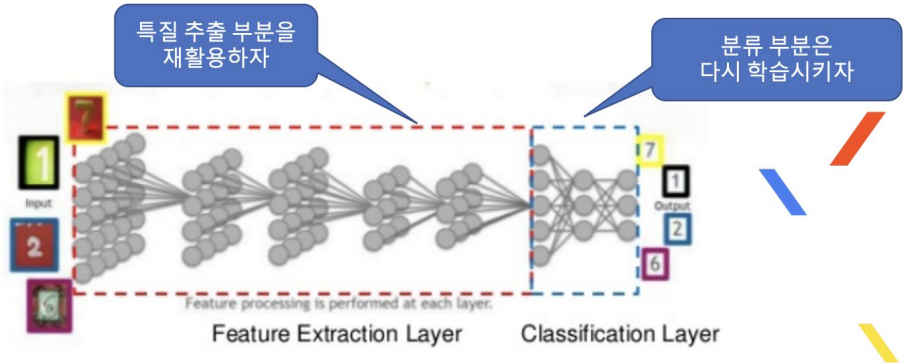
Transfer learning

CNN 기반 딥러닝 모델 훈련에 많은 양의 데이터 필요.
충분히 큰 데이터 세트 준비는 어려움.
전이 학습은 이를 해결.

전이학습은 아주 큰 데이터 세트를 사용해 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것을 말하며, 아주 큰 데이터 세트를 사용하여 훈련된 모델을 사전 훈련된 모델(네트워크)라고 한다.

즉, 결과적으로 비교적 적은 수의 데이터를 가지고도 우리가 원하는 과제를 해결할 수 있음

문제A --->(해결) 문제 A를 해결하면서 얻은 지식 및 경험 --->(적용과 전이학습) 문제 B

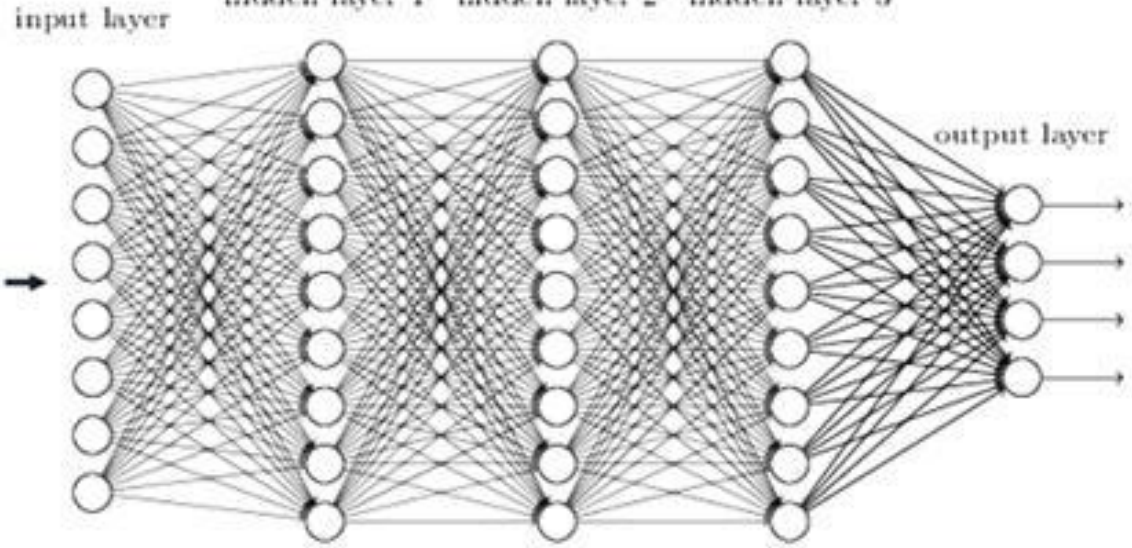


Deep neural networks learn hierarchical feature representations



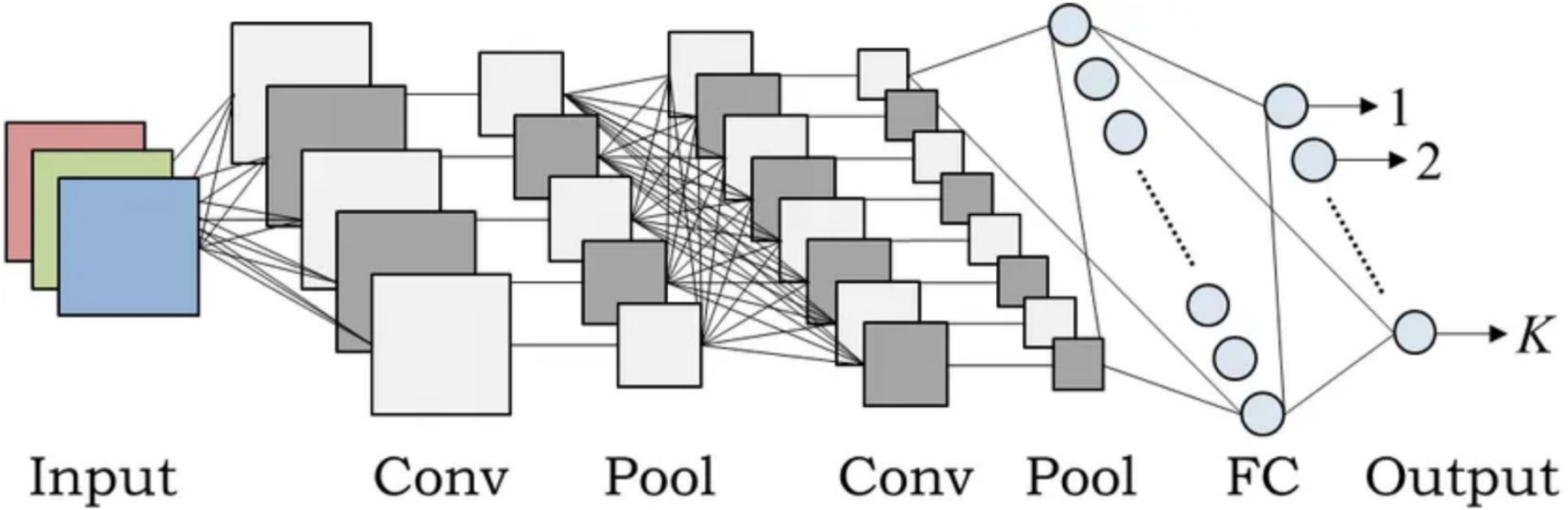
hidden layer 1 hidden layer 2 hidden layer 3



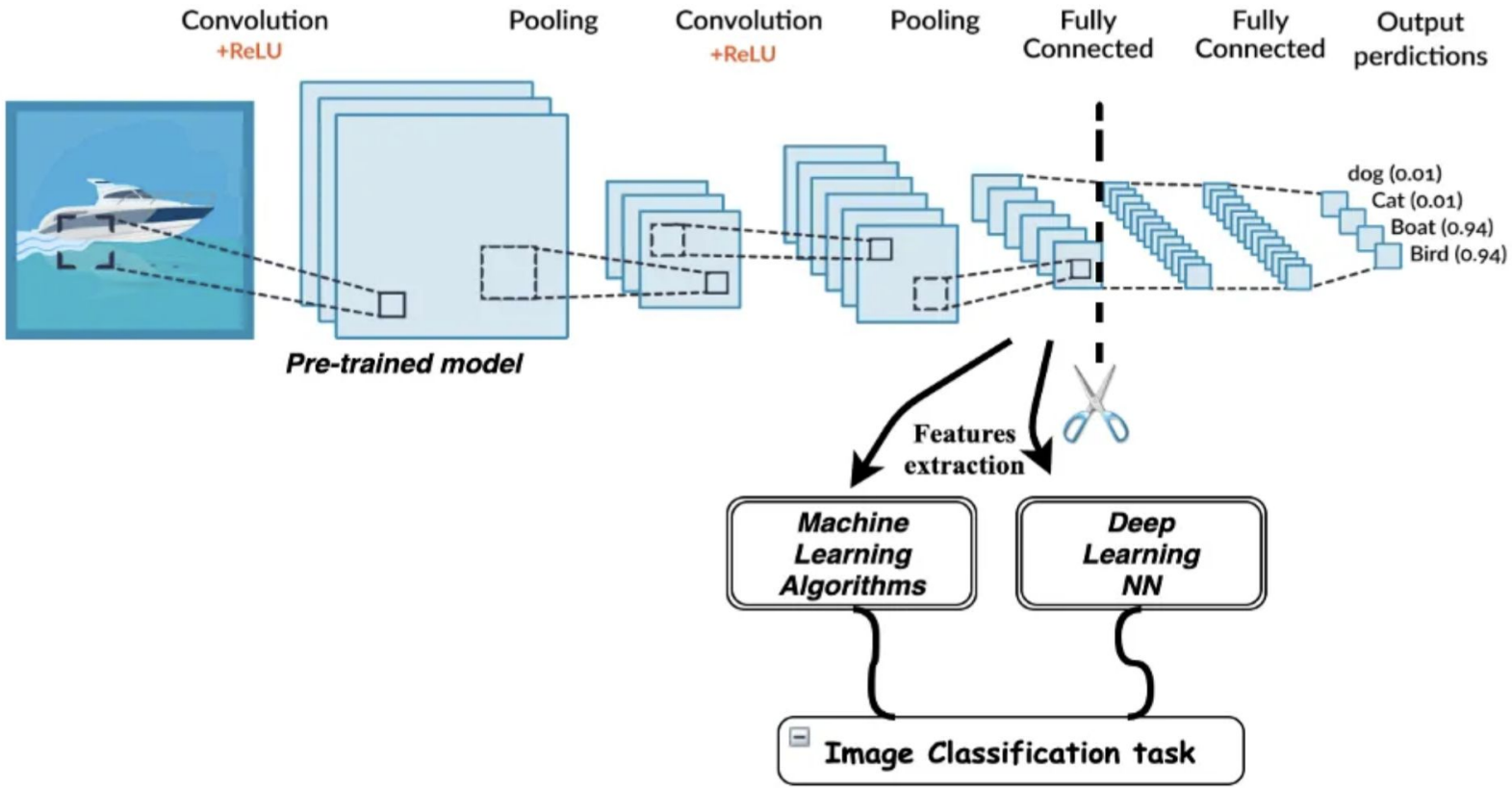


기존모델의 이미지분류 CNN 구조

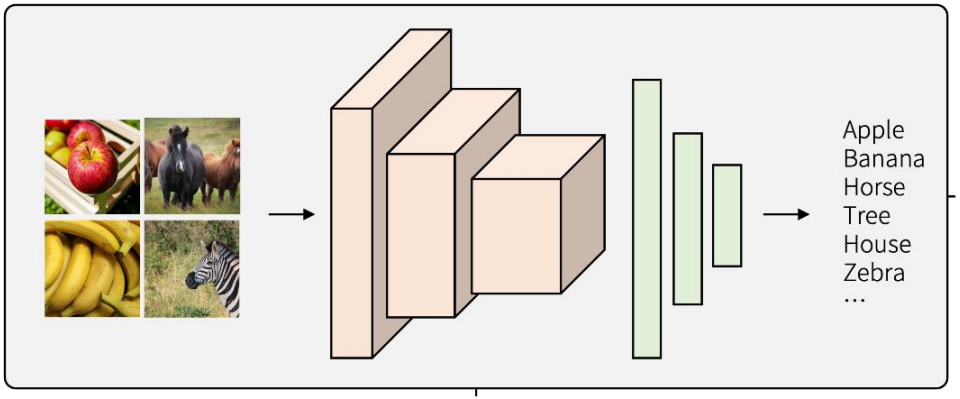
신경망 구조는 ConV레이어, Pool레이어, FC레이어,활성함수레이어로 총 4개로 이루어져 있음.



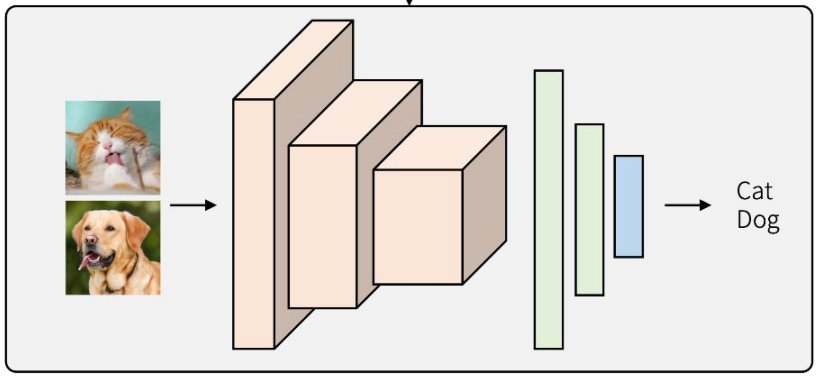
Transfer Learning(전이학습) - 기존 모델의 신경망에서 학습된 feature 에 마지막레이어 부분을 새로 학습



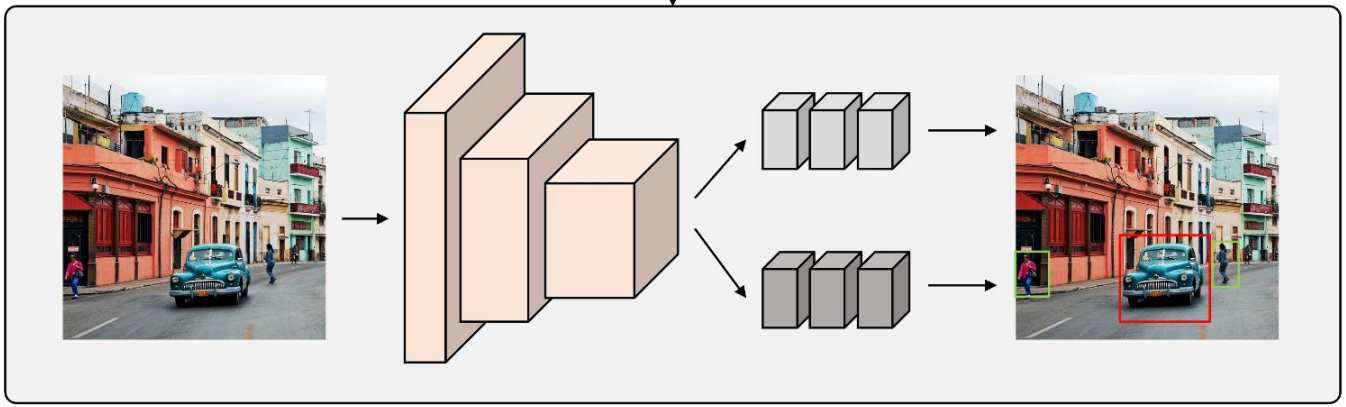
Classification



Classification



Object detection



전이학습의 실제 사용 사례

이미지 분류

ImageNet과 같은 대규모 데이터셋으로 사전 훈련된 컨볼루션 신경망(CNN) 모델을 사용하여 특정 종류의 동식물을 분류할 수 있습니다. 이러한 모델들은 이미 복잡한 이미지 특징을 학습했기 때문에, 새로운 이미지 분류 작업에 적용할 때 더 빠른 학습과 높은 정확도를 기대할 수 있습니다.

자연어 처리

BERT나 GPT와 같은 대규모 언어 모델은 감정 분석, 텍스트 분류, 질문 응답 시스템 등 다양한 자연어 처리 작업에 활용될 수 있습니다. 이 모델들은 광범위한 언어 데이터로부터 학습되어, 새로운 언어 처리 작업에 효과적으로 적용될 수 있습니다.

전이학습에 사용될 기존 학습완료 모델 성능 비교 [예측시간, 정확도, 크기 (메모리 사용량)]

